

Understanding Tree Model Dominance in Tabular Learning - Performance, Robustness, and Stability Across Classical, Deep and Foundation (pre-trained) Models

Suvendu Barai*

Communication Systems and Networks

Technische Hochschule Köln

Köln, Germany

suvendu.barai@smail.th-koeln.de

Abstract—Tree-based models remain highly competitive on tabular learning tasks despite the rapid progress of deep learning. In this work, we present a systematic comparison of tree-based, neural, and recent pretrained tabular models across a diverse benchmark of classification and regression datasets. Our study goes beyond clean-data evaluation by combining three perspectives: predictive performance after hyperparameter optimization, robustness under controlled data challenges, and stability under covariate shift. We evaluate classical tree ensembles (Random Forest, XGBoost, LightGBM, and CatBoost), neural baselines (MLP and TabNet), and recent tabular foundation-style models (TabPFN and APT) on six datasets, including two regression variants of the Diamonds dataset with different correlation-filtering thresholds. To assess robustness, we introduce missingness and synthetic high-cardinality categorical features, and we analyze clean versus shifted test distributions using both predictive degradation and SHAP-based stability measures. Across the selected benchmark, boosted tree methods show the most consistent empirical pattern: they usually remain close to the best clean-data performance while also preserving stronger robustness and feature-attribution stability than the neural baselines. Neural models remain competitive in selected cases, but in this benchmark they are generally more sensitive to feature-space changes, hyperparameter choice, and attribution instability. Pretrained tabular models reduce part of this gap, yet they do not consistently surpass boosted tree methods. These findings suggest that the continued strength of tree-based models in this study is linked not only to predictive accuracy, but also to robustness and stability under controlled tabular stress conditions. The results should therefore be interpreted as evidence from a controlled benchmark rather than as a universal claim.

Index Terms—tabular machine learning, tree-based models, deep learning, hyperparameter optimization, robustness, covariate shift, SHAP, feature attribution stability

I. INTRODUCTION

I-A. Background and Motivation

Tabular data is one of the most common data formats in applied machine learning and appears in domains such as finance, healthcare, logistics, and industrial analytics. Although deep learning has achieved major success in computer vision and natural language processing, tree-based methods remain strong baselines for many tabular prediction tasks. In practice, models such as Random Forest, XGBoost, LightGBM,

and CatBoost are often difficult to outperform, especially on datasets with mixed feature types, moderate sample sizes, and irregular data structure.

This continued strength raises an important question: why do tree-based models remain so effective on typical tabular data? A comparison based only on clean benchmark accuracy is not sufficient to answer this. In realistic settings, models must also remain reliable under incomplete inputs, difficult categorical structure, and changes in test-time data distribution. In this study, we approximate these issues through controlled stress tests and covariate-shift perturbations; these experiments are intended as heuristic probes of model behavior, not as exhaustive simulations of all real deployment conditions.

II. RELATED WORK

Tree-based ensembles remain central baselines for supervised tabular learning. Random Forests introduced bagging-based tree ensembles as robust nonparametric predictors [1], while XGBoost, LightGBM, and CatBoost established highly effective gradient-boosting variants with different strengths in scalability, efficiency, and categorical-feature handling [2]–[4]. These methods remain important reference points because they often combine strong predictive accuracy, practical efficiency, and stable behavior on structured datasets.

Neural tabular models have attempted to close this gap through specialized architectures and representation learning. TabNet introduced attentive feature selection [5], while TabTransformer and SAINT use attention-based contextualization and contrastive pretraining [6], [7]. Gorishniy et al. showed that carefully tuned simple neural baselines and FT-Transformer improve substantially over earlier tabular neural networks, but do not consistently displace boosted tree ensembles [8]. Broader benchmarks such as TabZilla and TabArena further show that relative model performance depends strongly on dataset characteristics and evaluation protocol [9], [10]. Grinsztajn et al. provide a closely related interpretation, arguing that tree-based models are well matched to typical tabular data because of their inductive biases for feature orientation, uninformative variables, and irregular decision functions [11].

Recent pretrained tabular models provide another comparison point. TabPFN performs in-context tabular prediction using a transformer pretrained on synthetic tasks [12], while APT extends this direction through adversarially pretrained tabular prediction [13]. These models reduce part of the dataset-specific tuning burden, but their robustness and feature-stability behavior under perturbation and shift remain less established than their clean benchmark performance.

Robustness-oriented tabular evaluation has also become increasingly relevant. TableShift studies distribution shift in tabular data [14], Gardner et al. analyze subgroup robustness and show that tree-based methods are strong empirical baselines [15], and TabularBench evaluates adversarial robustness for tabular deep learning [16]. Compared with these works, our study focuses on a complementary controlled setting combining missingness, synthetic high-cardinality stress, covariate-shift evaluation, and feature-attribution stability within one tree-versus-neural-versus-pretrained comparison.

For feature-stability analysis, we use SHAP as a diagnostic tool for comparing attribution consistency under perturbation and shift, not as evidence of causal feature use [17]. For rank-based model comparison, we follow Friedman and Nemenyi-style classifier-comparison methodology [18]. Hyperparameter optimization is performed with Optuna [19]. The benchmark datasets are drawn from commonly used public tabular sources, including Titanic, Telco Churn, DryBean, the scikit-learn digits dataset, and Diamonds [20]–[24].

III. EXPERIMENTAL SETUP

III-A. Datasets

We selected six tabular datasets spanning both classification and regression tasks in order to evaluate model behavior across different feature structures, target types, and levels of learning difficulty. The classification benchmark consists of the Titanic, Telco Churn, DryBean, and tabularized MNIST datasets, while the regression benchmark consists of two curated variants of the Diamonds dataset: diamonds v2 and diamonds v3.

Each dataset also plays a distinct structural role in the benchmark. Titanic represents a small mixed-feature binary classification problem in which limited sample size and heterogeneous attributes can favor models with strong inductive bias. Telco Churn provides a medium-scale binary classification task with many categorical predictors, making it particularly relevant for studying categorical handling and sensitivity to nuisance structure. DryBean is a multiclass benchmark with primarily numerical shape descriptors, allowing us to assess model behavior in a structured geometric setting. Tabularized MNIST serves as a high-dimensional numerical classification task, where the advantages of deep models may become more pronounced than in smaller mixed-type datasets. The file used in this project is not the original 70k-sample MNIST dataset. It is the full 8×8 tabular digits benchmark, which naturally contains 1,797 samples and 64 features.

The two Diamonds variants were included to study regression behavior under different levels of feature redundancy control. Both are derived from the same source dataset, but they differ in the correlation threshold used during feature

TABLE I: Summary of datasets used in this study.

Dataset	Task	Samples	Feat.	Num/Cat	Note
diamonds v2	Reg.	219,703	24	6/18	Pearson 0.85
diamonds v3	Reg.	219,703	22	4/18	Pearson 0.70
drybean	Multi-cls.	4,970	12	12/0	Bean benchmark
mnist	Multi-cls.	1,797	64	64/0	Tabular digits
telco	Binary cls.	7,032	20	4/16	Churn dataset
titanic	Binary cls.	183	11	6/5	Survival task

filtering. In diamonds v2, features were filtered using a Pearson correlation threshold of 0.85, whereas diamonds v3 used a stricter threshold of 0.70, removing more moderately correlated predictors. This distinction allows us to assess whether stronger decorrelation affects model behavior differently, particularly for neural models that may respond more sensitively to changes in feature redundancy.

The benchmark is intentionally heterogeneous but moderate in scale. It is not designed to isolate dataset-size effects systematically, and it does not include a controlled learning-curve or subsampling study across sample sizes. We therefore interpret dataset-size effects cautiously and treat such analysis as future work. A summary of the datasets, including task type, number of samples, and feature composition, is provided in Table I.

III-B. Models

We evaluated three groups of tabular learning models: tree-based methods, neural tabular models, and pretrained tabular foundation-style models.

The tree-based group includes Random Forest (RF), XGBoost, LightGBM, and CatBoost. These models were selected because they represent the strongest and most widely used classical approaches for tabular prediction. Random Forest serves as a robust bagging-based ensemble baseline, while XGBoost, LightGBM, and CatBoost provide modern gradient-boosted tree frameworks with different design choices for efficiency, split construction, and categorical feature handling.

The neural group includes a standard multilayer perceptron (MLP) and TabNet. The MLP was included as a simple dense neural baseline, whereas TabNet was chosen as a tabular-specific neural architecture based on sequential attention and feature selection. These two models allow comparison between a general-purpose neural approach and a specialized deep tabular model.

In addition, we included two recent pretrained tabular models, TabPFN and APT. Unlike the other models, these methods are designed for zero-shot or low-tuning inference using pretrained representations rather than extensive dataset-specific optimization. They were therefore evaluated using their default pretrained configurations instead of the full hyperparameter optimization procedure. Their inclusion broadens the benchmark beyond conventional neural baselines and allows us to assess whether recent pretrained tabular models narrow the gap to tree-based methods.

A compact summary of the evaluated models is given in Table II.

This creates an intentional but important asymmetry in the comparison. RF, XGBoost, LightGBM, CatBoost, MLP,

TABLE II: Summary of evaluated model families and tuning strategy.

Model	Family	Task	HPO	Note
RF	Tree	Cls./Reg.	Yes	Bagging ensemble
XGBoost	Tree	Cls./Reg.	Yes	Gradient boosting
LightGBM	Tree	Cls./Reg.	Yes	Histogram boosting
CatBoost	Tree	Cls./Reg.	Yes	Native categorical support
MLP	Neural	Cls./Reg.	Yes	Dense feed-forward network
TabNet	Neural	Cls./Reg.	Yes	Sequential attention model
TabPFN	Pretrained	Cls.	No	Default pretrained config
APT	Pretrained	Cls.	No	Fixed pretrained checkpoint

and TabNet are dataset-specifically tuned, whereas TabPFN and APT are evaluated as pretrained classification models using their available default or fixed-checkpoint configurations. Consequently, the pretrained-model results should not be interpreted as a fully symmetric HPO comparison. Instead, they indicate how these models behave in the practical low-tuning setting for which they are commonly used.

III-C. Hyperparameter Optimization Protocol

We applied hyperparameter optimization (HPO) separately for each dataset–model pair using Optuna. For the tunable models, each optimization run used a fixed budget of 30 trials, and the best configuration was selected according to validation performance before final evaluation on the held-out test set.

Optuna used the Tree-structured Parzen Estimator (TPE) sampler for sequential model-based hyperparameter search. Each trial sampled one candidate configuration and trained the model on the training split. For classification tasks, the validation objective was F1-score; for regression tasks, the objective was R^2 . The final test score was computed only once after selecting the best validation configuration. Unless otherwise stated, HPO was stopped after the fixed 30-trial budget rather than by convergence.

The same trial budget was used for the tunable model families to keep the protocol simple and comparable. However, equal trial counts do not guarantee equal search fairness because model families differ in the number, type, and sensitivity of their hyperparameters. The 30-trial budget should therefore be interpreted as a bounded practical HPO setting rather than an exhaustive search.

For tree-based models, the search spaces included parameters such as the number of trees, maximum depth, learning rate, subsampling, column sampling, and regularization terms, depending on the model family. For neural models, we tuned architectural and optimization parameters including the number of layers, hidden dimensions, dropout, learning rate, batch size, and, for TabNet, architecture-specific parameters such as decision and attention dimensions. The exact model-specific search spaces, number of tuned hyperparameters, objectives, and stopping criteria are summarized in Table III

III-D. Repeated-seed uncertainty

To quantify sensitivity to random initialization and data splitting, the final evaluation was repeated across three seeds (0, 42, and 123) using the selected HPO configuration for

each model–dataset pair. We report the mean and standard deviation of the main test metric across successful repeated runs. Classification datasets are summarized with macro-F1 for multiclass tasks and F1 for binary tasks, whereas regression datasets are summarized with R^2 Table IV. This analysis is intended as a lightweight uncertainty check rather than a full nested cross-validation study; pretrained/default baselines whose randomness is not controlled through the same HPO fitting process are reported with explicit notes.

III-E. Robustness stress scenarios

These perturbations were applied to four datasets—diamonds v2, Telco, DryBean, and tabularized MNIST—selected to cover regression, binary and multiclass classification, and both mixed-type and purely numerical feature settings. Titanic was excluded because it already contains substantial intrinsic missingness and has a small sample size, making additional synthetic stress less clean to interpret. diamonds v3 was excluded because it is a structural variant of diamonds v2, so including both would introduce unnecessary duplication at this stage.

The perturbation levels were chosen as moderate heuristic stress tests rather than domain-calibrated simulations. Their purpose is to expose model-family differences under controlled feature-space degradation, not to reproduce a specific deployment process.

For the missingness scenario, we randomly removed 15% of feature values from 20% of randomly selected columns in both the training and test sets. The goal was to simulate moderate but nontrivial incompleteness under a controlled and reproducible perturbation level. Missing values were then handled through each model’s preprocessing pipeline. Numerical features were imputed using a central-tendency rule, while categorical features were assigned either the most frequent level or a dedicated “Missing” category, depending on the model family and preprocessing design. After this transformation, each model was retrained using the same hyperparameter configuration selected during the clean-data HPO stage and then evaluated on the correspondingly perturbed test set.

For the high-cardinality scenario, we introduced two synthetic categorical variables, each containing 150 unique categories, into both the training and test data. These variables were generated at random and were not intended to carry predictive signal. Their purpose was to simulate nuisance categorical structure of the kind frequently encountered in tabular applications, such as ID-like or highly sparse categorical fields. The perturbed datasets were then processed using the encoding strategy appropriate to each model family. Tree-based methods received ordinal-style categorical representations, while neural models used the corresponding expanded representation required by their preprocessing pipeline. As in the missingness experiment, all models were retrained on the augmented data using the best clean-data hyperparameters rather than retuned specifically for the perturbation.

This setting tests whether models can ignore random nuisance categorical structure; it does not evaluate the case where high-cardinality variables contain genuine predictive information.

TABLE III: Hyperparameter optimization search spaces. Classification models were optimized for F1-score, using macro-F1 for multiclass datasets; regression models were optimized for R^2 . Optuna used a TPE sampler with seed 42.

Model	# HPs	Search space	Objective	Stopping
RF	4	$n_estimators \in [120, 400]$, $max_depth \in [4, 18]$, $min_samples_split \in [2, 10]$, $min_samples_leaf \in [1, 4]$	F1 / R^2	Fixed trial budget
LightGBM	6	$n_estimators \in [150, 400]$, $num_leaves \in [15, 255]$, $max_depth \in [-1, 16]$, $learning_rate \in [10^{-3}, 0.2]$ log, $subsample \in [0.6, 1.0]$, $colsample_bytree \in [0.6, 1.0]$	F1 / R^2	Early stopping 50 rounds
CatBoost	4	$iterations \in [200, 500]$, $learning_rate \in [10^{-3}, 0.2]$ log, $depth \in [4, 10]$, $l2_leaf_reg \in [10^{-2}, 10]$	F1 / R^2	Validation best model
XGBoost	5	$n_estimators \in [150, 400]$, $max_depth \in [3, 10]$, $learning_rate \in [10^{-3}, 0.2]$ log, $subsample \in [0.6, 1.0]$, $colsample_bytree \in [0.6, 1.0]$	F1 / R^2	Fixed trial budget
MLP	4	$h1 \in [64, 256]$, $h2 \in [32, 128]$, $alpha \in [10^{-5}, 10^{-2}]$ log, $learning_rate_init \in [10^{-4}, 10^{-2}]$ log	F1 / R^2	200 epochs, early stopping
TabNet	9	$n_d, n_a \in \{8, 16, \dots, 64\}$, $n_steps \in [3, 8]$, $gamma \in [1, 2]$, $lambda_sparse \in [10^{-6}, 10^{-2}]$ log, $momentum \in [0.01, 0.4]$, $learning_rate \in [10^{-3}, 5 \times 10^{-2}]$ log, $batch_size \in \{256, 512, 1024\}$, $virtual_batch_size \in \{128, 256, 512\}$	F1 / R^2	Patience 20

TABLE IV: Repeated-seed uncertainty for final selected configurations.

Model	Metric	Mean	Std.	Runs
CatBoost	F1	0.801	0.148	12
XGBoost	F1	0.797	0.145	12
LightGBM	F1	0.782	0.156	12
MLP	F1	0.769	0.157	12
TabNet	F1	0.662	0.259	12
RF	F1	0.642	0.396	12
CatBoost	R^2	0.818	0.030	6
MLP	R^2	0.793	0.093	6
RF	R^2	0.776	0.025	6
XGBoost	R^2	0.774	0.062	6
LightGBM	R^2	0.717	0.114	6
TabNet	R^2	0.021	0.124	6

Pretrained/default baselines are excluded because repeated random initialization is not comparable for fixed pretrained configurations. Regression experiments used a deterministic 10,000-row cap. Runs denote verified dataset-model-seed evaluations.

For each robustness scenario, we recorded predictive performance, training time, and model size. For classification tasks, robustness was assessed using Accuracy and F1-score; for regression tasks, we report R^2 , RMSE, and MAE where appropriate.

III-F. Covariate shift and SHAP stability protocol

Beyond explicit robustness stress tests, we also investigated model behavior under covariate shift. This analysis was conducted on the three classification datasets—Titanic, Telco, and DryBean—because they together cover binary and multiclass classification, mixed feature structures, and semantically interpretable predictors, making SHAP-based stability comparisons more meaningful. We did not include the regression datasets because this stage was designed around joint evaluation of Accuracy/F1 degradation and feature-attribution stability in classification settings. Tabularized MNIST was excluded because its image-derived feature space is less aligned with the

feature-semantics questions emphasized in the present shift analysis.

For each dataset, models were trained on the original training data and then evaluated on two versions of the test set: the original clean test set and a shifted test set constructed through controlled perturbations of both numerical and categorical features. Only input features were perturbed; labels were kept unchanged. Thus, the protocol approximates covariate shift rather than concept drift. For numerical variables, the shifted test set was created by applying a small mean shift, additive Gaussian noise, and a small fraction of outlier injections. Concretely, numerical features received a mean shift of 0.2σ , additive Gaussian noise with standard deviation 0.5σ , and 2% outlier perturbations of magnitude 3σ , where σ denotes the feature-specific standard deviation. For categorical variables, 10% of values were replaced with the most frequent category and an additional 5% were replaced with a previously unseen category. This protocol was intended to simulate moderate distributional drift without changing the underlying target definition. SHAP-based stability is used as a relative diagnostic of attribution consistency, not as evidence of causal feature use.

III-G. Robustness significance testing

For robustness metrics, we compare clean and perturbed scores at the dataset level. For each model and perturbation type, we compute paired clean-versus-perturbed differences using the same datasets whenever both values are available. Where sufficient paired observations are available, we apply the Wilcoxon signed-rank test to the clean and perturbed F1/Accuracy scores. Because the number of robustness datasets is small, these tests are interpreted descriptively rather than as definitive evidence. We therefore use them to support the robustness discussion, but avoid strong claims from non-significant differences.

III-H. Computational environment and resource reporting

All experiments were executed on a local Apple MacBook Pro workstation with an Apple M2 chip, 8 CPU cores, an integrated 10-core Apple GPU, and 8 GB RAM, running macOS 26.2 - 26.5.2. The software environment used Python 3.12.4, scikit-learn 1.5.0, LightGBM 4.6.0, XGBoost 3.0.2, CatBoost 1.2.8, PyTorch 2.7.1, TensorFlow 2.19.0, Optuna 4.3.0, and TabPFN 6.3.2. APT was used from the local official repository checkout under external/APT; the package did not expose a numeric version identifier.

CUDA was not available on the local machine. Although Apple Metal Performance Shaders (MPS) was available, the training scripts used CPU execution for models whose device selection relied on CUDA availability. Runtime was measured as wall-clock time using the same preprocessing and evaluation pipeline for all models. HPO search time and final-model training time were recorded separately where available. For pretrained models, external pretraining cost was not included.

IV. HYPERPARAMETER OPTIMIZATION RESULTS

IV-A. Cross-dataset performance comparison

We begin by comparing model performance after hyperparameter optimization across the full benchmark. Because the study includes both classification and regression tasks, raw metric values are not directly comparable across datasets. We therefore aggregate results using within-dataset ranks together with summary statistics such as wins, top-2 frequency, and average gap to the best model.

Within this benchmark, Table V: shows that the strongest boosted tree methods occupy the most favorable region of the comparison. This is important because it indicates not only occasional wins, but repeated near-best placement across heterogeneous tasks. In other words, their advantage is not driven by one unusually favorable dataset or one isolated hyperparameter outcome.

The three leading boosted tree models all achieve overall mean ranks in the narrow range of roughly 2.2 to 2.7, with average gaps to the best model remaining below about 6%. The neural models show a different profile. They can be competitive on selected datasets, but their overall placement is less stable and their average gaps are wider. This suggests that neural performance is more localized: strong results are achievable, but less consistently across datasets and less robustly across configurations. The practical implication is that the tree-based advantage is not only about peak score, but also about repeatability across the benchmark.

The pretrained models provide a useful contrast. TabPFN reduces part of the gap to the tuned neural baselines, especially on some classification tasks, but it does not approach the cross-dataset consistency of the strongest boosted trees. APT is clearly weaker in this benchmark. Taken together, the results support a recurring empirical pattern in favor of boosted tree models within the selected benchmark, but they should not be read as a definitive ranking of all tabular model families.

In addition to the aggregated ranks in Table V, Table VI reports individual dataset-level scores, showing which datasets drive the average ranking.

IV-B. Classification and regression performance patterns

Across the classification datasets, tree-based methods remain the most consistently competitive family. Their advantage is especially visible on smaller or mixed-type problems, where strong inductive bias and selective use of informative predictors appear to matter more than representational flexibility alone. On such tasks, the strongest tree models do not merely win by a narrow margin; they frequently maintain a clear buffer over the neural baselines, indicating that the difficulty is not simply insufficient tuning of deep models, but a deeper mismatch between model inductive bias and the structure of the tabular task.

At the same time, the classification results are not uniform across all datasets, and this variation is informative. On tasks that are more homogeneous or more strongly numerical, the gap between trees and neural models narrows. This suggests that the tree advantage is not absolute, but depends on the extent to which the dataset rewards selective splitting on stable predictors rather than high-capacity distributed representations. In other words, the benchmark does not support the simplistic claim that neural models always underperform on tabular data; instead, it suggests that they become more competitive when the input structure is more aligned with the strengths of dense representation learning.

The regression results reinforce this interpretation while adding an important structural contrast through diamonds v2 and diamonds v3. Because both variants are derived from the same source dataset but differ in correlation-filtering threshold, they allow us to observe how stronger redundancy control affects different model families. The benchmark indicates that tree-based methods remain comparatively stable under this change, whereas neural models are more sensitive to the removal of moderately correlated predictors.

This Diamonds contrast is particularly relevant given the broader motivation of the paper. One of the recurring questions in tabular learning is whether tree-based models benefit from their ability to preserve feature orientation and exploit informative thresholds without depending heavily on distributed representations. The relative stability of tree models across diamonds v2 and diamonds v3 is consistent with that view. Neural models, by contrast, may benefit more from feature redundancy or moderately correlated support variables, and therefore exhibit greater sensitivity when that redundancy is filtered more aggressively.

IV-C. Train-test gap and overfitting behavior

To assess overfitting behavior, we computed a train-test gap for each dataset-model pair. For classification datasets, the gap is defined as

$$\Delta_{\text{gap}} = F1_{\text{train}} - F1_{\text{test}},$$

and for regression datasets as

$$\Delta_{\text{gap}} = R^2_{\text{train}} - R^2_{\text{test}}.$$

Larger positive values indicate stronger overfitting.

TABLE V: HPO model summary across datasets. Ranks are computed within each dataset first, then averaged across datasets. Lower is better for rank/gap/time/size; higher is better for Wins and Top-2. Avg. Gap to Best (%) is computed per dataset and then averaged per model. For each dataset, we take the best score among all models (F1 for classification, R^2 for regression) as the reference, compute each model’s percentage shortfall from that reference, and then average these gaps across datasets. Thus, lower values indicate better predictive quality, and 0% means the model matches the dataset-best score.

Model	Mean Rank (Cls.)	Mean Rank (Reg.)	Mean Rank (Overall)	Wins	Top-2	Avg. Gap to Best (%)	Time Rank	Size Rank
xgboost	2.25	2.00	2.17	1	4	6.09	2.83	3.50
catboost	2.00	3.50	2.50	2	3	5.40	4.67	2.00
lgbm	1.75	4.50	2.67	2	3	4.80	3.00	2.83
rf	4.00	1.00	3.00	3	3	18.78	3.67	4.83
tabnet	3.50	5.50	4.17	2	2	8.83	7.00	5.00
mlp	6.00	4.50	5.50	0	0	13.29	5.17	3.00
tabpfn	5.75	–	5.75	0	0	16.04	5.50	6.75
apt	7.75	–	7.75	0	0	40.11	1.50	8.00

TABLE VI: Individual dataset-level test scores after HPO. Classification datasets report F1; regression datasets report R^2 . NA indicates that the corresponding model was not evaluated for that task.

Dataset	Task	Metric	RF	XGBoost	LightGBM	CatBoost	MLP	TabNet	TabPFN	APT
diamonds_v2	Regression	R^2	0.99997	0.99997	0.98646	0.99524	0.99186	0.98754	NA	NA
diamonds_v3	Regression	R^2	0.99998	0.99998	0.99516	0.99500	0.94392	0.82606	NA	NA
drybean	Multiclass classification	F1	0.952	0.968	0.987	0.986	0.766	0.839	0.874	0.661
mnist	Multiclass classification	F1	1.000	1.000	1.000	1.000	0.997	1.000	0.989	0.942
telco	Binary classification	F1	0.000	0.613	0.697	0.607	0.607	0.884	0.597	0.000
titanic	Binary classification	F1	0.909	0.960	0.941	1.000	0.806	0.806	0.809	0.784

For Telco, RF and APT obtain F1 = 0.000 because they predict only the majority non-churn class. Binary F1 is computed with respect to the positive churn class; therefore, majority-class prediction can yield nonzero accuracy but zero F1.

TABLE VII: Train–test gap for final selected post-HPO models. Classification scores use F1; regression scores use R^2 . Gaps are computed as train score minus test score.

Model	Train	Test	Gap	n
MLP	0.876	0.852	0.024	6
LightGBM	0.920	0.906	0.014	4
TabNet	0.892	0.882	0.009	4
RF	0.720	0.715	0.005	4
CatBoost	0.903	0.898	0.005	4
XGBoost	0.863	0.860	0.002	3

n denotes the number of verified dataset–model pairs included in the average. Larger positive gaps indicate stronger overfitting.

IV-D. Predictive quality versus computational and model-size trade-offs

Predictive performance alone does not fully determine model suitability for tabular learning. Figure 1 therefore summarizes the average training time and model size across datasets.

The results indicate that stronger performance does not always require disproportionate computational cost, but neither is there a simple monotonic relation between accuracy and efficiency.

A practically important pattern is that the strongest tree-based methods often sit near a favorable performance–efficiency frontier. Their predictive advantage is therefore reinforced by operational convenience: they are not only strong in score, but also competitive in training cost and storage requirements. Neural models, in contrast, may require greater computational burden or larger effective representations while

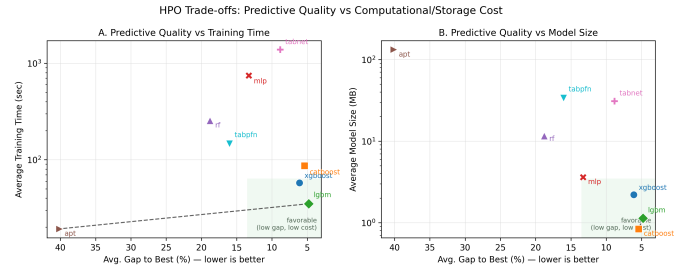


Fig. 1: Predictive quality versus computational and storage cost after hyperparameter optimization. Panel A compares average gap to the best model with training time, and Panel B compares average gap to the best model with model size. Models closer to the lower-right region combine stronger predictive quality with lower cost. For pretrained models, the reported time excludes external pretraining and refers only to dataset-level evaluation/setup time in the benchmark pipeline.

still exhibiting less stable benchmark-wide ranking. This does not make them uncompetitive, but it does reduce their attractiveness when predictive gains are small or inconsistent. The pretrained models occupy an intermediate position: they reduce dataset-specific tuning burden, but this convenience does not automatically translate into superior overall predictive or efficiency trade-offs.

IV-E. Statistical ranking across classification datasets

To complement the raw classification results, we compared model ranks across the four classification datasets using a

Friedman test followed by Nemenyi post-hoc analysis at $\alpha = 0.05$. The resulting critical-difference diagram should be interpreted cautiously: its main message is not a strict total ordering of models, but the extent to which observed rank differences are statistically distinguishable.

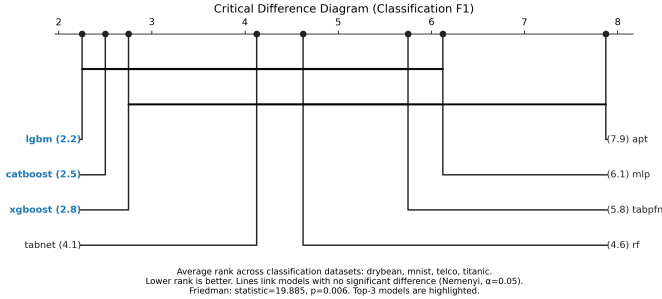


Fig. 2: Critical difference diagram for classification datasets based on F1-score ranks.

In this benchmark, several pairwise differences are not statistically significant, even when the mean-rank gaps appear visually large. This is plausible for two reasons. First, the classification suite is small, containing only four datasets, which limits statistical power. Second, the Nemenyi procedure is conservative, especially when many models are compared simultaneously. In our case, the combination of eight models, a small number of datasets, and several tied or extreme outcomes increases the critical difference and makes broad non-significance unsurprising.

The diagram therefore supports a restrained conclusion. LightGBM and CatBoost occupy the strongest part of the ranking, while APT lies clearly on the weak end. Beyond those broad separations, however, many intermediate pairwise differences should not be over-interpreted. Thus, the rank-based analysis is most useful here as evidence that the strongest tree-based models remain concentrated near the top of the classification benchmark, rather than as proof of a finely resolved hierarchy among all methods.

V. ROBUSTNESS TO MISSINGNESS AND HIGH-CARDINALITY FEATURES

V-A. Robustness under missingness

The missingness results indicate that tree-based models are comparatively stable when inputs are made incomplete. The key pattern in Table VIII is not only that their average predictive degradation is small, but that this is accompanied by strong feature-retention behavior. The tree-based models generally preserve high SHAP overlap under missingness, indicating that they continue to rely on largely similar core predictors even after values are removed. This is an important distinction: missingness robustness is more convincing when performance retention and attribution stability move together rather than in isolation.

Among all evaluated models, CatBoost presents the strongest overall profile. Its mean predictive degradation remains close to zero, and its SHAP overlap stays near the top of the benchmark. LightGBM and Random Forest are

similarly stable, though slightly weaker in attribution retention. By contrast, the neural models are less consistent. MLP shows a moderate predictive drop together with lower top-feature stability, suggesting that it remains more sensitive to disruption of the input space. TabNet is closer to the tree models in average predictive retention, but its feature-stability profile is weaker, indicating that comparable score retention does not necessarily imply equally stable internal reliance on features.

The efficiency statistics reinforce this interpretation. Under missingness, reduced training cost should not automatically be interpreted as robustness. In several cases, faster training appears to reflect a changed effective training regime rather than a genuinely stable response to incomplete inputs. Overall, the missingness experiment suggests that, in the evaluated datasets, tree-based models tend to preserve both predictive behavior and feature-attribution structure more consistently under degraded inputs.

V-B. Robustness under high-cardinality categorical stress

Table VIII shows that tree-based models remain comparatively stable when irrelevant high-cardinality categorical features are added. XGBoost and CatBoost show particularly stable combined profiles, with small predictive changes and high SHAP overlap. LightGBM and Random Forest also remain comparatively robust, although their feature-retention scores are slightly weaker than those of the strongest boosted models.

The neural models are more affected by this stress condition. MLP shows the clearest deterioration, combining weaker attribution stability with a large increase in training time. TabNet retains stronger predictive performance than MLP, but its SHAP overlap and Jaccard values remain below those of the tree-based models.

Overall, the high-cardinality experiment suggests that tree ensembles are better able to ignore uninformative categorical nuisance structure in the evaluated benchmark. This supports the interpretation that their robustness advantage is linked not only to predictive retention, but also to more stable feature reliance under controlled feature-space clutter.

VI. FEATURE IMPORTANCE AND STABILITY UNDER COVARIATE SHIFT

Covariate shift probes a different form of robustness from the missingness and high-cardinality experiments. Here the issue is not incomplete inputs or added nuisance variables, but whether models trained on the original data remain stable when the test distribution changes while the task itself remains fixed. We evaluate this stability along two complementary axes: predictive retention and feature-importance retention. Table IX reports the corresponding averages across the three shift datasets.

VI-A. Predictive stability under shift

The shift results show that clean-data strength and shifted-data stability are related, but not identical. CatBoost, LightGBM, and XGBoost retain the highest shifted accuracy values,

TABLE VIII: Model-level robustness summary under missingness and high-cardinality stress. Values are averaged over the available robustness datasets. Performance deltas are measured relative to the clean condition, so values closer to zero indicate greater predictive robustness; positive values indicate improvement under perturbation. Training-time change is reported as percentage change relative to clean training time. SHAP overlap is the average number of shared features in the top-10 feature sets between clean and perturbed conditions, and Jaccard is the corresponding top-10 set overlap ratio. NA denotes unavailable values, primarily because training-time or SHAP-based stability quantities were not computed for APT and TabPFN.

Model	Δ Perf. M	Δ Perf. H	Δ Time% M	Δ Time% H	Overlap M	Overlap H	Jaccard M	Jaccard H
CatBoost	-0.0105	-0.0081	-12.23	1.11	9.00	9.25	0.826	0.871
LightGBM	0.0005	-0.0154	1.97	2.12	9.00	8.50	0.818	0.742
Random Forest	0.0011	-0.0112	0.43	15.44	9.00	8.25	0.839	0.710
XGBoost	-0.0045	-0.0050	-4.67	15.03	9.00	9.00	0.826	0.826
MLP	-0.0085	-0.0214	-7.88	493.18	7.00	5.00	0.617	0.361
TabNet	0.0364	0.0740	-39.68	-46.62	6.25	5.50	0.537	0.422
APT	0.0124	0.0152	NA	NA	NA	NA	NA	NA
TabPFN	-0.0107	-0.0628	NA	NA	NA	NA	NA	NA

M = missingness, H = high-cardinality.

TABLE IX: Average predictive and SHAP-based stability under covariate shift. Values are averaged over the three shift datasets. Δ values are computed as shifted minus clean, so values closer to zero indicate smaller predictive degradation. SHAP ρ denotes mean Spearman rank correlation and SHAP Jac. denotes mean Jaccard overlap of the top-10 features. n_{perf} and n_{SHAP} indicate the number of datasets contributing to the predictive and SHAP aggregates, respectively. NA indicates unavailable SHAP-based quantities.

Model	Acc. Clean	Acc. Shift	Δ Acc.	F1 Clean	F1 Shift	Δ F1	SHAP ρ	SHAP Jac.	n_{perf}	n_{SHAP}
CatBoost	0.933	0.716	-0.217	0.864	0.621	-0.243	0.992	0.939	3	3
LightGBM	0.919	0.715	-0.204	0.875	0.616	-0.259	1.000	0.939	3	3
RF	0.850	0.704	-0.146	0.620	0.474	-0.146	0.989	0.879	3	3
XGBoost	0.908	0.710	-0.198	0.847	0.613	-0.234	NA	NA	3	0
MLP	0.748	0.677	-0.071	0.727	0.637	-0.090	0.943	0.698	3	3
TabNet	0.819	0.696	-0.123	0.843	0.671	-0.172	0.948	0.467	3	3
TabPFN	0.799	0.627	-0.172	0.788	0.562	-0.226	NA	NA	3	0
APT	0.638	0.593	-0.044	0.481	0.403	-0.079	NA	NA	3	0

while TabNet attains the highest shifted F1-score. This indicates that the ordering depends on the metric used. Smaller degradation alone should also not be interpreted as better robustness, because models such as MLP or APT start from lower clean baselines and therefore have less performance to lose.

The more meaningful interpretation is therefore joint rather than absolute. A strong shifted model should retain useful performance after shift while also maintaining a competitive absolute score. On this criterion, the boosted tree models remain strong in shifted accuracy, whereas TabNet remains competitive in shifted F1. The results therefore suggest model-family differences, but not a single universal winner across all shift metrics.

VI-B. Feature-importance stability under shift

The SHAP analysis adds a second layer by asking whether models continue to rely on similar dominant predictors after shift. Among the models with available SHAP values, CatBoost, LightGBM, and Random Forest show the strongest attribution stability. CatBoost and LightGBM both achieve high top-feature Jaccard overlap, while Random Forest also preserves a comparatively stable feature ranking. TabNet and MLP retain weaker top-feature stability, indicating that com-

petitive shifted performance does not necessarily imply stable feature reliance.

VII. CROSS-EXPERIMENT ANALYSIS

VII-A. Rank stability versus robustness

A central cross-experiment question is whether clean-data strength carries over to more demanding evaluation settings. The results suggest that it does, but only partially. The strongest clean benchmark models are also the ones most likely to remain competitive under missingness, nuisance categorical structure, and covariate shift. This indicates that clean-data rank is not arbitrary: it contains real information about broader model quality. At the same time, the relationship is not exact. Strong clean performance does not automatically imply equal robustness, and robustness itself depends on which kind of perturbation is introduced. This is precisely the relationship illustrated by Figure 3.

What matters most here is consistency across evaluation regimes. The strongest tree-based models are not simply top-ranked on the clean benchmark; they remain near the top when the evaluation moves from standard prediction to controlled stress and distributional displacement. This makes their advantage qualitatively different from isolated wins on individual datasets. It suggests that their strength in this benchmark is not

only an isolated clean-score pattern: the same model family often remains competitive when the feature space becomes incomplete, cluttered, or shifted.

This perspective also helps clarify why some non-tree models appear promising but do not alter the broader conclusion. Neural and pretrained methods can reach competitive results in selected settings, yet their relative position is less stable once the benchmark is widened beyond clean accuracy alone. Thus, the main lesson of the combined experiments is not merely that tree-based models often rank well, but that their behavior remains more consistent under the additional evaluation criteria used here.

VII-B. Performance drop versus attribution drift

A second cross-experiment pattern is that predictive degradation and attribution drift are related, but clearly not interchangeable. Models that lose predictive quality under perturbation often also show weaker feature-stability behavior, yet the two effects do not move in lockstep. A model can preserve a broadly similar explanatory structure while still suffering a substantial performance drop, and it can also retain reasonable predictive quality while shifting which features it relies on most. For this reason, neither predictive metrics nor SHAP-based stability should be treated as a complete proxy for the other.

This distinction is especially important for interpretation. Predictive robustness tells us whether the model still performs well after the data changes. Attribution stability asks a different question: whether the model still appears to trust the same predictors. The latter matters because two models with similar shifted performance may differ strongly in how stable their internal reliance pattern remains. In practical terms, a model that preserves score only by reallocating importance across features is a weaker form of robustness than one that remains anchored to the same core signals.

The cross-experiment evidence therefore supports a more careful definition of robustness. A robust tabular model should not only resist predictive degradation, but should do so while preserving a coherent and stable explanatory structure. On this combined criterion, the strongest tree-based models remain the most convincing, because they repeatedly show that predictive retention and feature-priority retention can coexist rather than trade off sharply against one another.

VIII. WHY TREE-BASED MODELS REMAIN STRONG ON TABULAR DATA

The explanations in this section are interpretive and based on consistency between the observed results and known tabular-model behavior; they should not be read as direct causal proof of the mechanisms.

VIII-A. Selective feature usage under nuisance structure

A first explanation is that the strongest tree-based models are more selective in how they use features. The robustness experiments show that, when the input space is

made more difficult through missingness or synthetic high-cardinality categorical features, the tree models generally retain both stronger predictive performance and stronger feature-importance stability. This is clearest under high-cardinality stress. CatBoost shows only a small mean performance change, while preserving high SHAP overlap and the highest mean Jaccard among the evaluated models. By contrast, the MLP degrades more strongly in predictive terms and exhibits a much larger drop in attribution stability, indicating that the added nuisance structure is not simply ignored but enters the learned representation more materially.

This difference is consistent with the way the model families treat uninformative structure. Tree ensembles perform feature selection implicitly through splitting: a feature that does not improve the objective need not be used at all. High-cardinality nuisance variables therefore create less harm when they fail to produce useful gain. In contrast, neural models often absorb such structure into a dense representation or a larger encoded input, which increases optimization burden even if the added variables carry no genuine signal. The earlier project analyses already described this as a form of distraction by nuisance structure, and the current efficiency results support the same interpretation: under high-cardinality stress, MLP exhibits a very large increase in training time and model size, whereas the strongest tree models remain comparatively stable.

The same logic helps explain why some neural or foundation models did not perform better even when their clean-data baselines were competitive. The issue is not that they never capture useful signal; rather, they appear less able to suppress structurally difficult but uninformative inputs once those inputs enter the representation pipeline. This is one reason why strong clean-data performance does not always translate into equally strong behavior under perturbation.

VIII-B. Attribution stability and reliance on core predictors

A second explanation is that tree-based models preserve a more stable notion of which features matter. This is one of the most important findings of the paper, especially given the emphasis on core predictors, feature types, and target-relevant structure.

Under covariate shift, CatBoost and Random Forest attain the highest mean SHAP stability, with near-perfect mean Spearman correlations and clearly higher Jaccard overlap than MLP or TabNet. The same pattern appears under missingness and high-cardinality stress, where tree models retain roughly eight to nine of their top ten features on average, while neural models preserve fewer. Figure 3 summarizes this relationship directly: the tree-based models cluster closer to the region combining stronger performance retention and higher attribution stability, whereas neural models tend to move downward more strongly in feature-stability space.

This matters because robust prediction and stable explanation are not identical. A model can retain moderate predictive performance while changing which predictors it emphasizes most strongly; conversely, it can preserve the same core features while still suffering performance degradation if the

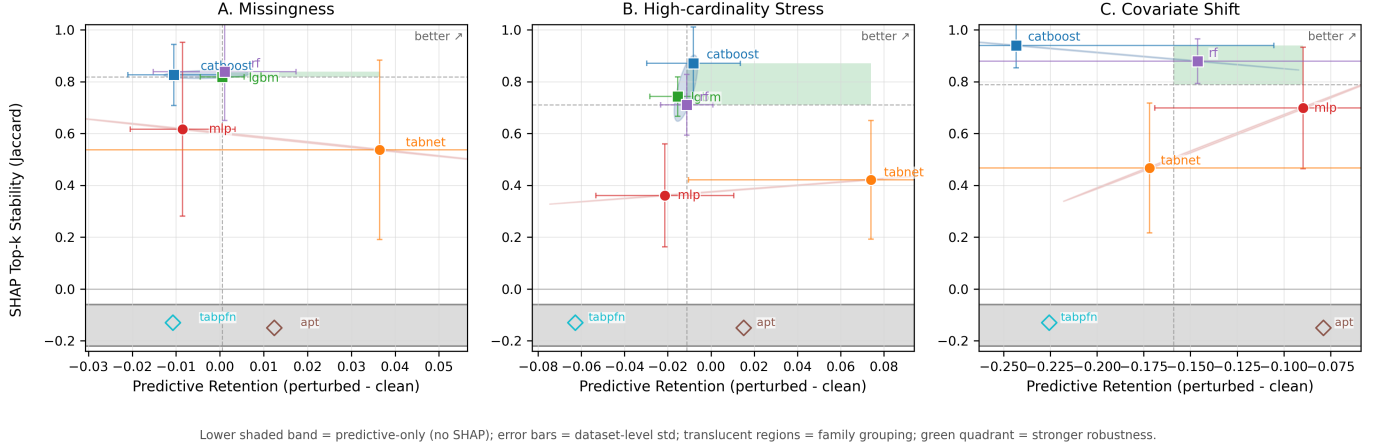


Fig. 3: Performance retention versus attribution stability under three perturbation types: missingness, high-cardinality stress, and covariate shift. Each point denotes a model. The x-axis reports predictive retention relative to the clean condition, and the y-axis reports SHAP top- k stability measured by Jaccard overlap. Error bars indicate dataset-level standard deviation where available. The lower shaded band marks models for which predictive results are available but SHAP-based stability values were not computed. Translucent regions indicate broad model-family groupings, stronger robustness corresponds to smaller predictive change (x-axis values closer to zero) together with higher attribution stability.

shifted distribution weakens their utility. The strongest tree-based models are more compelling because they tend to preserve both. In that sense, they are not merely more accurate predictors, but also more stable estimators of the underlying explanatory structure. This aligns closely with the feature-centered interpretation developed in the earlier deep drafts: tree models appear to keep focusing on the most informative variables, whereas neural models are more likely to reallocate importance under perturbation or shift.

This also helps explain why current neural and pretrained tabular models did not consistently surpass the tree baselines. Even when competitive in score, they were less consistent in preserving the same feature-priority structure. A model whose dominant predictors drift more under shift is harder to trust as a stable tabular learner, particularly in applications where feature semantics matter and explanations must remain interpretable across deployment conditions.

VIII-C. Feature orientation, redundancy, and irregular target structure

A third explanation concerns the structural character of tabular data itself. The datasets in this study were deliberately chosen to vary in feature composition, target type, cardinality, sample size, and correlation structure. Across these settings, the strongest tree-based models appear better aligned with three recurring properties of tabular data: heterogeneous feature spaces, moderate redundancy, and irregular decision structure.

The earlier project analyses emphasized that tree models preserve the original orientation of the feature space more naturally than neural models. This point should not be stated as a theorem for the present benchmark, but it is consistent

with the observed results. Split-based models evaluate individual features or simple partitions directly, whereas neural models construct hidden representations through learned linear combinations. When the task depends on threshold-like rules, sparse informative columns, or a small set of dominant predictors, the tree bias can be advantageous. This interpretation is supported by the shift analyses, where tree models often retained the same top variables under perturbation, and by the Diamonds comparison, where stronger decorrelation affected neural models more than tree ensembles.

The NeurIPS benchmark paper frames this more generally as a question of tabular inductive bias: tree-based methods appear better suited to preserving feature orientation, handling uninformative variables, and learning irregular functions, whereas neural networks are often biased toward smoother solutions and less well-matched invariances [11]. Our results are consistent with that view. We do not directly replicate their transformation-based experiments, but our missingness, high-cardinality, and shift analyses point in the same direction. The strongest tree models remain stable when the feature space is partially degraded, expanded with nuisance structure, or distributionally displaced. That is difficult to explain by optimization alone; it suggests an underlying match between tree-based decision structure and the statistical form of many tabular problems.

VIII-D. Why neural and pretrained tabular models still fall short

The present results do not imply that neural or pretrained tabular models are uniformly weak. In fact, they remain competitive in selected settings, and the pretrained models reduce part of the gap relative to standard neural baselines on some classification tasks. However, in this benchmark they do

not match the strongest tree-based models consistently across the combined criteria.

For the neural models, the main limitation is not simply insufficient capacity. The pattern across experiments suggests a more specific failure mode: they are more sensitive to feature-space changes, more dependent on favorable hyperparameter configurations, and more likely to redistribute importance across predictors when the input distribution changes. The earlier analyses described this as a tendency to overfit or mis-prioritize features under nuisance perturbation, and the current robustness and shift results support that reading. Even after bounded hyperparameter tuning, MLP and TabNet remain less stable than the strongest tree models either in predictive terms, attributional terms, or both.

For the pretrained models, the picture is slightly different. Their main strength is not full dominance over tree ensembles, but reduction of the gap to conventional neural models with less dataset-specific tuning. Even so, they do not consistently surpass the strongest tree-based methods. APT remains weak in overall benchmark ranking, and both APT and TabPFN lack the same breadth of SHAP-based stability evidence available for the core models. More importantly, where predictive data are available, they still do not establish a new robust frontier beyond the boosted tree baselines. Thus, pretraining helps, but does not remove the core tabular inductive-bias problem identified in the literature and reflected in our benchmark.

There is also a practical asymmetry in the comparison. The tree methods used here are ensembles, and ensembling contributes regularization and variance reduction with manageable cost. The neural baselines, by contrast, are single models. In principle, neural ensembles could improve performance, but at substantially higher computational cost. This does not invalidate the comparison, because in practice the relevant comparison is between the strongest widely usable representatives of each paradigm. On that criterion, tree ensembles remain more attractive not only because they perform strongly, but because they do so with less tuning fragility and better robustness-efficiency trade-offs.

IX. DISCUSSION AND CONCLUSION

IX-A. Limitations and Future Prospects

Although the study compares clean performance, robustness stress, and covariate-shift stability across diverse tabular settings, several limitations remain:

- **Benchmark scope:** The benchmark covers different task types and feature structures, but remains moderate in size. The robustness analysis uses four datasets and the covariate-shift analysis uses three datasets, so the findings should be read as recurring empirical patterns rather than universal claims.
- **Controlled perturbations:** Missingness, high-cardinality stress, and covariate shift were implemented through designed perturbation protocols. These settings isolate specific failure modes, but real deployment shifts may be more complex, temporal, or domain-specific.
- **Interpretability scope:** SHAP-based stability measures whether models preserve similar feature-attribution pat-

terns, but it does not establish causal feature use or complete interpretability. Future work could include local, interaction-level, or alternative attribution analyses.

- **Incomplete model coverage:** Predictive covariate-shift results are now available for the main tree models, including LightGBM and XGBoost. However, SHAP-based stability was unavailable for XGBoost, APT, and TabPFN. Therefore, explanatory-stability claims are restricted to models with both predictive and SHAP outputs.
- **Bounded HPO protocol:** The same Optuna trial budget gives a simple comparison protocol, but equal trial counts do not guarantee equal effective tuning effort across model families. Larger budgets, repeated tuning runs, and more extensive seed analysis could change some rankings.
- **Uncertainty quantification:** The study includes a lightweight repeated-seed uncertainty analysis, but does not perform full nested cross-validation, confidence intervals, bootstrap uncertainty, or formal significance testing for all robustness metrics.
- **Future directions:** Future work should extend the benchmark with probabilistic quality, calibration, uncertainty estimation, repeated validation, and a more systematic analysis of neural or pretrained tabular embeddings for downstream tasks.

IX-B. Conclusion

This paper examined tabular model behavior beyond clean benchmark ranking by combining hyperparameter-optimized evaluation with robustness stress tests and feature-stability analysis. Within the selected benchmark, tree-based models, especially boosted tree ensembles such as CatBoost, LightGBM, and XGBoost, showed the most consistent empirical pattern across clean performance, controlled perturbations, covariate shift, and attribution stability.

The results suggest that the continued strength of tree-based models in this study is linked not only to predictive accuracy, but also to their ability to preserve useful behavior under controlled feature-space stress. Neural and pretrained tabular models remain competitive in selected cases, but their performance and stability were less consistent under the combined evaluation criteria used here.

These conclusions should not be read as a universal proof of tree-model dominance. Rather, the study supports a more careful evaluation message: tabular models should be compared not only by clean test accuracy, but also by robustness, feature-attribution stability, computational behavior, and the limits of the evaluation protocol itself.

REFERENCES

- [1] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [2] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2016, pp. 785–794.
- [3] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "Lightgbm: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/6449f44a102fde848669bdd9eb6b76fa-Paper.pdf

- [4] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "Catboost: Unbiased boosting with categorical features," in *Advances in Neural Information Processing Systems*, vol. 31, 2018. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2018/file/14491f3fca7f18f8d7cdc714c304b7f5-Paper.pdf
- [5] S. O. Arik and T. Pfister, "Tabnet: Attentive interpretable tabular learning," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 8, pp. 6679–6687, 2021.
- [6] X. Huang, A. Khetan, M. Cvitkovic, and Z. Karnin, "Tabtransformer: Tabular data modeling using contextual embeddings," *arXiv preprint arXiv:2012.06678*, 2020. [Online]. Available: <https://arxiv.org/abs/2012.06678>
- [7] G. Somepalli, M. Goldblum, A. Schwarzschild, C. B. Bruss, and T. Goldstein, "Saint: Improved neural networks for tabular data via row attention and contrastive pre-training," *arXiv preprint arXiv:2106.01342*, 2021. [Online]. Available: <https://arxiv.org/abs/2106.01342>
- [8] Y. Gorishniy, I. Rubachev, V. Khrulkov, and A. Babenko, "Revisiting deep learning models for tabular data," in *Advances in Neural Information Processing Systems*, vol. 34, 2021, pp. 18 932–18 943. [Online]. Available: <https://proceedings.neurips.cc/paper/2021/hash/9d86d83f925f2149e9edb0ac3b49229c-Abstract.html>
- [9] D. McElfresh, S. Khandagale, J. Valverde, V. Prasad C, G. Ramakrishnan, M. Goldblum, and C. White, "When do neural nets outperform boosted trees on tabular data?" *arXiv preprint arXiv:2305.02997*, 2023. [Online]. Available: <https://arxiv.org/abs/2305.02997>
- [10] N. Erickson, L. Purucker, A. Tschalzev, D. Holzmüller, P. M. Desai, D. Salinas, and F. Hutter, "Tabarena: A living benchmark for machine learning on tabular data," *arXiv preprint arXiv:2506.16791*, 2025. [Online]. Available: <https://arxiv.org/abs/2506.16791>
- [11] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 507–520. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/hash/0378c7692da36807bdec87ab043cdadc-Abstract-Datasets_and_Benchmarks.html
- [12] N. Hollmann, S. Müller, K. Eggenberger, and F. Hutter, "Tabpfn: A transformer that solves small tabular classification problems in a second," in *International Conference on Learning Representations*, 2023. [Online]. Available: <https://openreview.net/forum?id=cp5PvcI6w8>
- [13] Y. Wu and D. L. Bergman, "Zero-shot meta-learning for tabular prediction tasks with adversarially pre-trained transformer," *arXiv preprint arXiv:2502.04573*, 2025. [Online]. Available: <https://arxiv.org/abs/2502.04573>
- [14] J. Gardner, Z. Popovic, and L. Schmidt, "Benchmarking distribution shift in tabular data with tableshift," *arXiv preprint arXiv:2312.07577*, 2023. [Online]. Available: <https://arxiv.org/abs/2312.07577>
- [15] J. Gardner, Z. Popović, and L. Schmidt, "Subgroup robustness grows on trees: An empirical baseline investigation," *arXiv preprint arXiv:2211.12703*, 2022. [Online]. Available: <https://arxiv.org/abs/2211.12703>
- [16] T. Simonetto, S. Ghamizi, and M. Cordy, "Tabularbench: Benchmarking adversarial robustness for tabular deep learning in real-world use-cases," *arXiv preprint arXiv:2408.07579*, 2024. [Online]. Available: <https://arxiv.org/abs/2408.07579>
- [17] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/8a20a8621978632d76c43dfd28b67767-Paper.pdf
- [18] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006. [Online]. Available: <https://www.jmlr.org/papers/v7/demsar06a.html>
- [19] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2019, pp. 2623–2631.
- [20] Kaggle, "Titanic: Machine learning from disaster," <https://www.kaggle.com/c/titanic>, 2012, accessed: 2026-06-20.
- [21] IBM Sample Data Sets, "Telco customer churn," <https://www.kaggle.com/datasets/blastchar/telco-customer-churn>, 2019, accessed: 2026-06-20.
- [22] M. Koklu and I. A. Ozkan, "Multiclass classification of dry beans using computer vision and machine learning techniques," *Computers and Electronics in Agriculture*, vol. 174, p. 105507, 2020.
- [23] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: Machine learning in python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: <https://www.jmlr.org/papers/v12/pedregosa11a.html>
- [24] H. Wickham, *ggplot2: Elegant Graphics for Data Analysis*. Springer, 2016, reference for the Diamonds dataset included in the ggplot2 package.